

Mobile Development

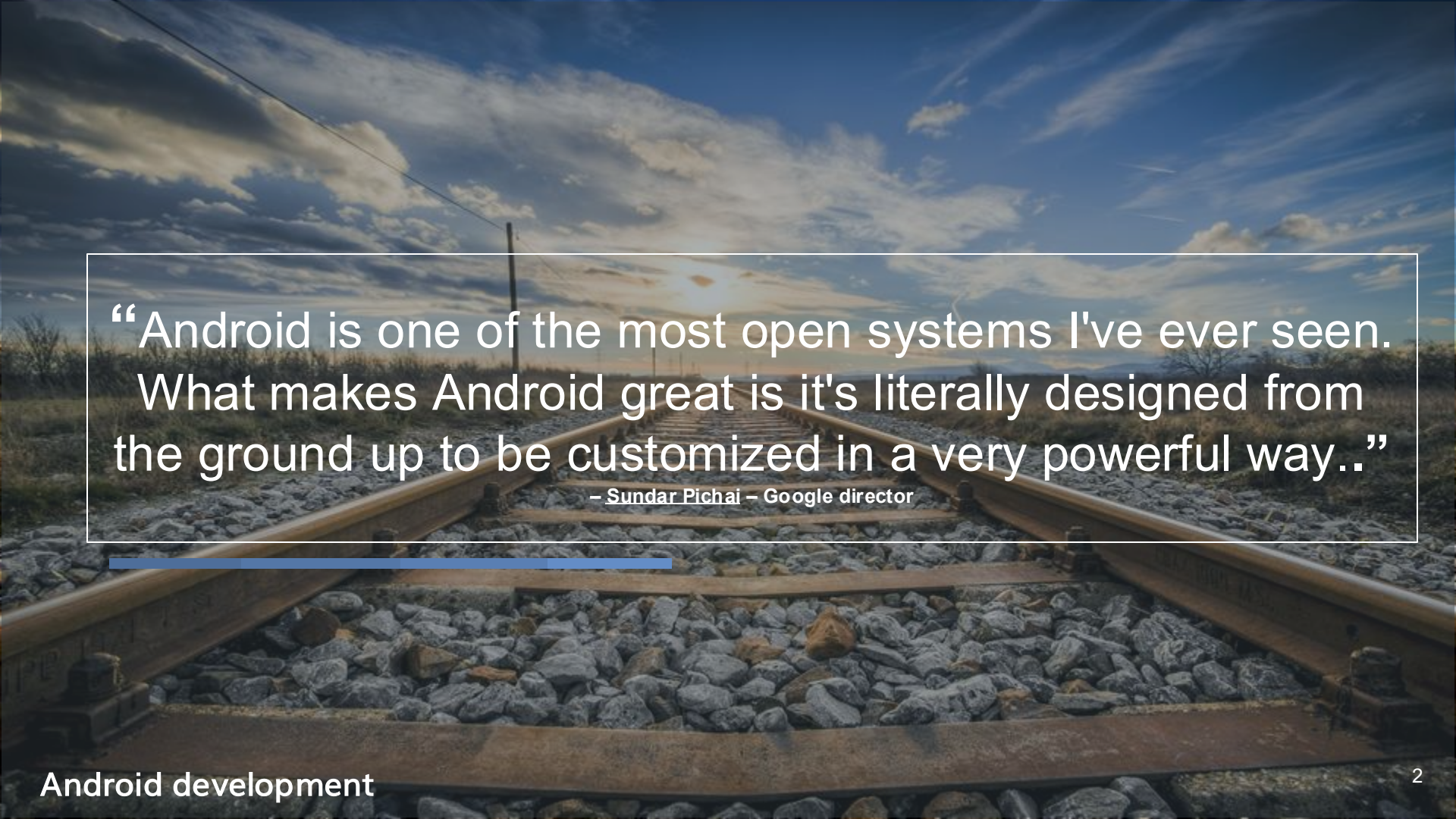
(Lab 5)

Instructor: Tran Vinh Khiem

March 1st, 2022



Smart Software System Team



“Android is one of the most open systems I've ever seen. What makes Android great is it's literally designed from the ground up to be customized in a very powerful way..”

– Sundar Pichai – Google director

Learning Objectives



- Provide you with realistic, hands-on experience developing Android applications.
- Create a portfolio of apps that you can show your friends, discuss in interviews, and borrow for other applications.

Task



Asynchronous

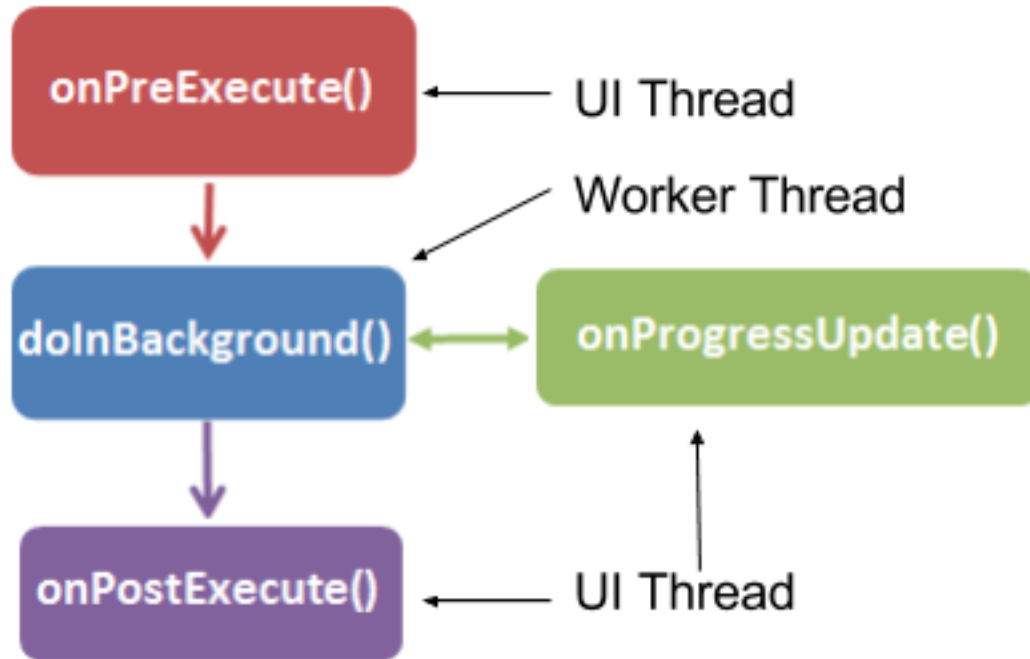
As you wait for the event to happen, you can do something else.

When an event happens, you will be called back, and you can act right away.

Synchronous

In this case, you'll have to wait for something to happen before you can do anything else.

AsyncTask



AsyncTask - Example



```
private class EtOEtTask extends AsyncTask <double, Void, int> {  
    @Override  
    protected void onPreExecute() {  
        // Perform setup - runs on user interface thread  
    }  
  
    @Override  
    protected int doInBackground(double...d) {  
        // Insert task here! This task will run in the background.  
    }  
  
    @Override  
    protected void onProgressUpdate(Void... param) {  
        // Provide progress information to the user or anything similar - operates on the UI thread  
    }  
  
    @Override  
    protected void onPostExecute(int result) {  
        // Update user interface - runs on the UI Thread  
    }  
}
```

To execute this code:

```
new EtOEtTask().execute();
```

AsyncTask - Exercise



- Do this codelab with Java and take screenshot
- https://google-developer-training.github.io/android-developer-fundamentals-course-practicals/en/Unit%203/71_p_create_an_asynctask.html



AsyncTaskLoader - Example

```
@Override
public Loader<DataType> onCreateLoader(int id, final Bundle args) {
    return new AsyncTaskLoader<DataType>(this) {
        String mCache;
        @Override
        protected void onStartLoading() {
            \\ Perform set up
        }
        @Override
        public String loadInBackground() {
            \\ Insert task here
        }
        @Override
        public void deliverResult() {
            \\ provide result here
        }
    };
}

@Override
public void onLoadFinished(Loader<String> loader) {
    // After tasks finish, what you want to do next? Show data? Show error?
}

@Override
public void onLoaderReset(Loader<String> loader) { \\ \\ When task needs to be reset}
```

To execute this code:
`loaderManager.initLoader();`
`loaderManager.restartLoader();`

AsyncTaskLoader – CodeLab - Exercise



- Do this codelab with Java and take screenshot
- <https://developer.android.com/codelabs/android-training-async-task-async-taskloader?index=..%2F..%2Fandroid-training#1>

Service



Foreground

Performs some operation that is noticeable to the user

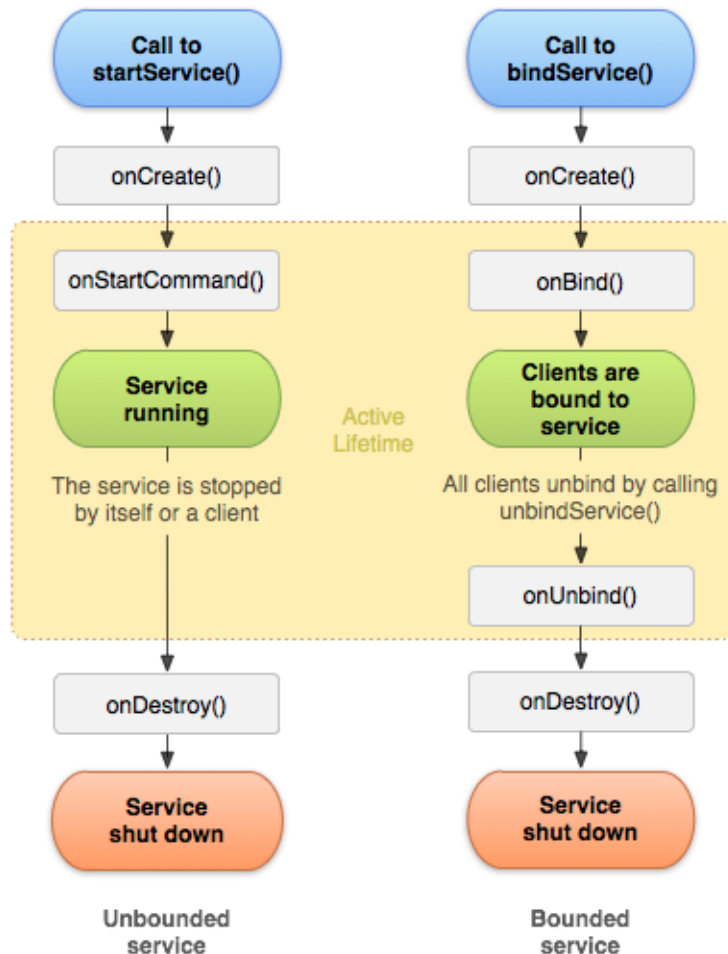
Background

performs an operation that isn't directly noticed by the user.

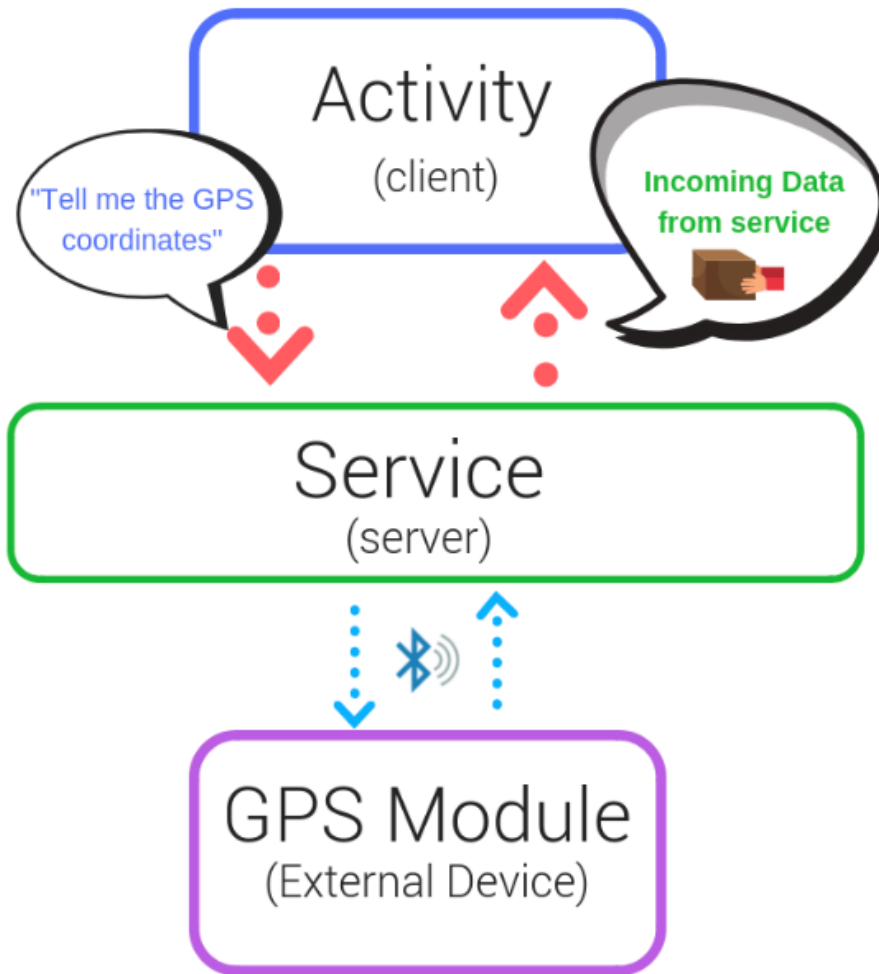
Bound

A service is *bound* when an application component binds to it by calling [`bindService\(\)`](#)

Service



Bound Service



Service



```
public class ExampleService extends Service {
    int startMode;        // indicates how to behave if the service is killed
    IBinder binder;        // interface for clients that bind
    boolean allowRebind; // indicates whether onRebind should be used

    @Override
    public void onCreate() {
        // The service is being created
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // The service is starting, due to a call to startService()
        return startMode;
    }

    @Override
    public IBinder onBind(Intent intent) {
        // A client is binding to the service with bindService()
        return binder;
    }

    @Override
    public boolean onUnbind(Intent intent) {
        // All clients have unbound with unbindService()
        return allowRebind;
    }

    @Override
    public void onRebind(Intent intent) {
        // A client is binding to the service with bindService(),
        // after onUnbind() has already been called
    }

    @Override
    public void onDestroy() {
        // The service is no longer used and is being destroyed
    }
}
```

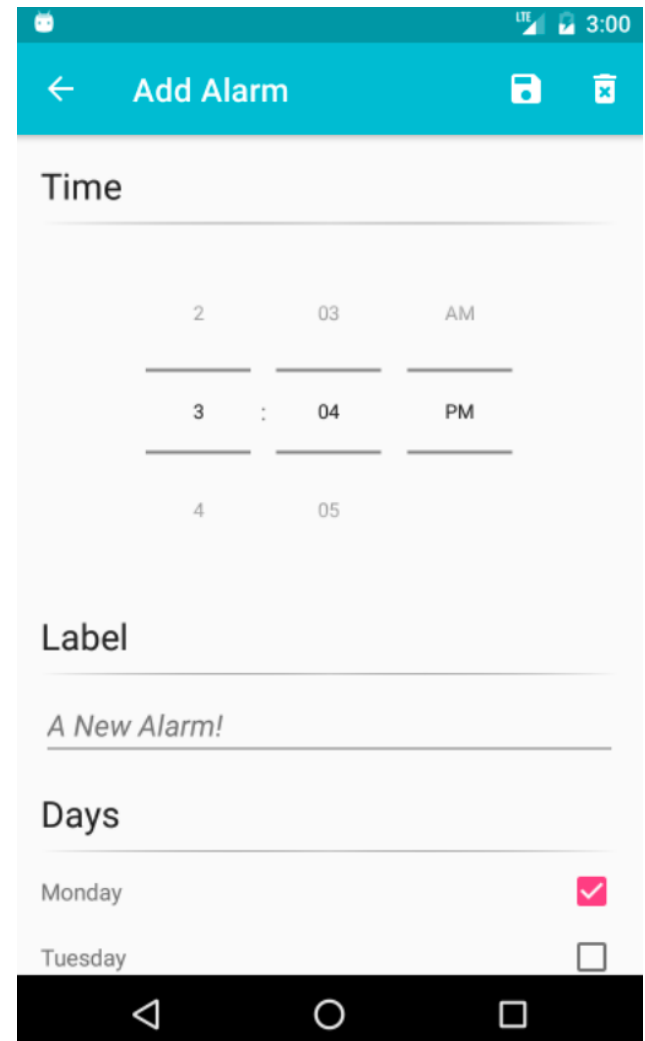
Recap exercise



- Create an Android app with the following features:
- A UI with three buttons: "Start Background Task", "Start Foreground Service", and "Bind to Service".
- Implement a Background Service to perform a simulated long-running task (e.g., counting numbers) and log the progress.
- Implement a Foreground Service to display a notification and perform a similar task.
- Implement a Bound Service to allow the Activity to interact with the service (e.g., get the current count).
- Use AsyncTask (for learning purposes) to perform a synchronous task and Coroutines for an asynchronous task, updating the UI with results.

Exercise

- Create Alarm application.
- Example of this app is on the picture.

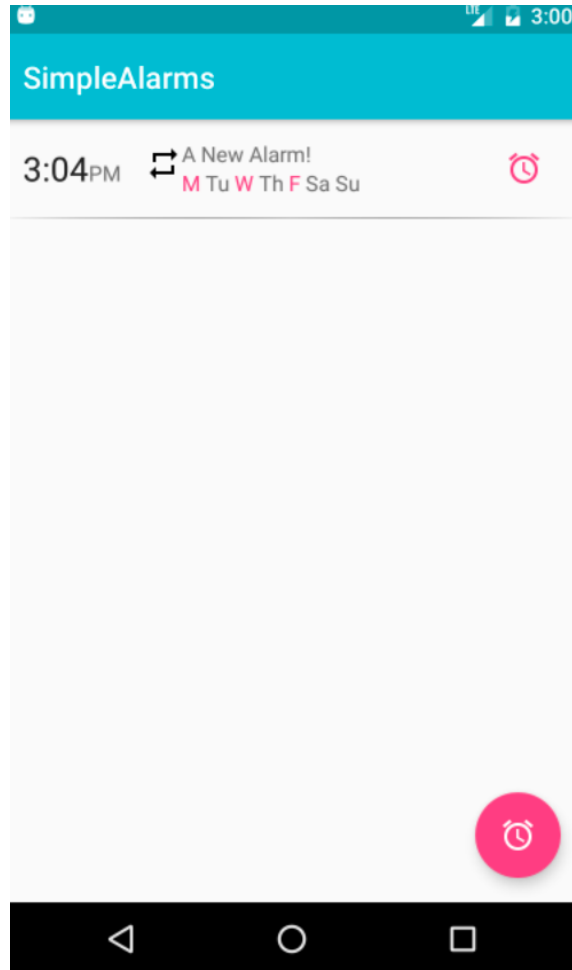


The screenshot shows an Android application titled "Add Alarm". The interface is divided into several sections:

- Time:** A digital clock display showing 2:03 AM. Below it, a second row shows 3:04 PM. At the bottom of this section, the numbers 4 and 05 are visible.
- Label:** A text input field containing the placeholder text "A New Alarm!".
- Days:** A list of days with checkboxes. "Monday" is checked (indicated by a red checkmark), and "Tuesday" is unchecked (indicated by an empty checkbox).

The app is running on an Android device, as evidenced by the status bar at the top (showing LTE, signal strength, and 3:00) and the navigation bar at the bottom (showing back, home, and recent apps buttons).

Exercise



Homework – Fitness app



- The Fitness Tracker App simulates step tracking with:
- A UI with vibrant interface with a quest theme, progress bars, badges for milestones (e.g., 10,000 steps), and animations for achievements.
- A Background Service: Periodically logs steps with simulated data, now storing them in a Room database for persistence.
- A Foreground Service: Displays a notification with real-time step counts and a motivational quote.
- A Bound Service: Allows the Activity to retrieve step counts and calories burned asynchronously, with a smooth UI update
- An AsyncTask: Calculates calories burned ($\text{steps} * 0.04$) and points earned ($\text{steps} / 100$) in the background.

Homework – Fitness app



- Syncs step counts and achievements to a WearOS watch face from previous lesson, showing a dynamic progress circle and badges.
- Make video demo for your app.
- Advanced feature: Send step, calories and points to Gemini API. Gemini will generate a workout suggestion (e.g., "Try a 20-min jog to boost your stamina") based on the input. Gemini creates tailored encouragement (e.g., "You're crushing it with 8,000 steps! Let's aim for 10,000!").



References

- <https://web.stanford.edu/>
- <https://developer.android.com/>
- <https://introcs.cs.princeton.edu/>
- <https://www.cs.utexas.edu/>
- <https://www.codepath.com/>



Thank you for listening

*"Coming together is a beginning;
Keeping together is progress;
Working together is success."*

- HENRY FORD